

阶段 3：复现与实现

图划分算法实现与复现结果

小组成员：姓名 1，姓名 2，姓名 3

CS240：算法设计与分析，2026 年春季学期

1 实现概览

本项目实现为一个小型 Python 工程。代码位于 `src/` 目录下，按照图生成、算法、评价指标、仿真、实验入口和绘图等功能拆分成多个模块。这样的模块化结构便于独立测试每个组件，也方便在同一实验流水线中比较多种图划分方法。

主要文件包括：

- `src/graphs.py`: 构造 benchmark 图；
- `src/algorithms.py`: 贪心、谱划分和 Kernighan–Lin 划分算法；
- `src/metrics.py`: 划分质量评价指标；
- `src/simulation.py`: 一致性平均仿真；
- `src/experiments.py`: 实验运行与 CSV 导出；
- `src/plotting.py`: 实验图像生成。

项目使用 Python、NumPy、NetworkX、pandas、matplotlib 和 pytest。所有报告中的实验均使用固定随机种子，因此结果可以复现。

2 实现的算法

2.1 贪心均衡划分

贪心基线逐个将顶点分配到分区中。顶点按照加权度数从高到低处理，使影响较大的顶点优先被放置。对于每个顶点，算法只考虑尚未达到容量上限的可行分区，并选择相对于已分配邻居产生最小增量割值的分区。容量设为近似 $\lceil n/k \rceil$ ，因此该方法可以保持接近完全均衡的分区大小。

该基线故意保持简单。它提供了快速比较点，有助于判断更复杂算法是否确实提升了划分质量。

2.2 递归谱二分

谱方法反复二分当前最大的分区，直到得到 k 个分区。对于一个子图，算法构造加权邻接矩阵 A 、度矩阵 D 和图拉普拉斯矩阵 $L = D - A$ 。随后计算 L 的特征向量，并使用 Fiedler 向量对顶点排序。排序后的顶点列表在中点处切分。

该方法很适合作为较强的初始划分。在网格图和随机几何图中，Fiedler 向量通常能够捕捉空间结构，因此谱二分往往能产生割值较低的平衡划分。

2.3 Kernighan–Lin k 路细化

实现中的 Kernighan–Lin 方法从谱划分得到的初始解出发。随后执行若干轮全局细化 pass。在每一轮中，算法遍历所有分区对。对于分区对 (a, b) ，取出当前属于这两个分区的顶点诱导子图，并以当前分配作为初始二分，调用 NetworkX 中稳定的 Kernighan–Lin 二分过程。

候选更新只有在不增加全局割边权重时才会被接受。该规则保证细化过程相对于割值是单调不恶化的。如果完整一轮全局 pass 之后目标函数没有继续下降，算法提前停止。

3 复现范围

原始 Kernighan–Lin 论文主要关注图二分。本项目复现的是其核心算法思想，而不是历史论文中的全部实验设置。具体复现内容包括：

1. 基于两个分区之间顶点交换的局部搜索细化；
2. 重复执行 pass，直到没有明显割值改进；
3. 将该方法作为细化过程，而不是仅作为随机初始划分器；
4. 实验测量割值改进和额外运行时间。

主要改写是 k 路划分设置。我们没有只做一次全局二分，而是在 k 路划分内部对分区对执行成对 Kernighan–Lin 细化。这更适合分布式优化应用，因为目标系统包含 k 个 worker，而不只是两个处理器。

4 Benchmark 实例

实现评估了四类图：

- **网格图**：具有强局部几何结构的图；
- **Erdos–Renyi 随机图**：无明显结构的随机图；
- **随机几何图**：具有空间局部连接特性的图；
- **Zachary karate club 图**：小型真实社交网络。

benchmark 共包含八个实例：

图类型	规模参数	$ V $	$ E $	k
grid	6	36	60	4
grid	8	64	112	4
grid	10	100	180	4
erdos_renyi	40	40	93	4
erdos_renyi	80	80	248	4
geometric	50	50	330	4
geometric	90	90	628	4
karate	34	34	78	2

5 评价指标

主要指标是割边权重：

$$\text{cut}(p) = \sum_{(u,v) \in E, p(u) \neq p(v)} w_{uv}.$$

割边权重越低，表示跨 worker 依赖越少或越便宜。

第二个指标是归一化割，它将每个分区边界权重除以该分区体积后求和。这有助于判断割值相对于各分区连接强度是否较小。

第三个指标是负载均衡比例：

$$\rho(p) = \frac{\max_i |\{v : p(v) = i\}|}{|V|/k}.$$

越接近 1 表示工作负载越均衡。

第四个指标是运行时间，单位为秒，用于衡量每种划分算法的计算代价。

最后，一致性仿真报告最终一致性误差和跨分区消息数量。该仿真保持原始通信图不变，因此不同划分不会改变收敛动力学；划分影响的是有多少消息需要跨 worker 边界传输。

6 复现结果

实验复现了 Kernighan–Lin 风格局部细化的预期定性行为。在所有 benchmark 上，平均割边权重为：

$$\text{greedy} = 98.25, \quad \text{spectral} = 72.75, \quad \text{Kernighan-Lin} = 66.88.$$

一致性仿真中的平均跨分区消息数为：

$$\text{greedy} = 15460, \quad \text{spectral} = 11360, \quad \text{Kernighan-Lin} = 10440.$$

Kernighan–Lin 细化方法取得了最低的平均割值和最低的平均跨 worker 通信量。在八个图实例中，它有六个实例取得最佳割值。这支持了预期复现目标：局部细化通常可以相比简单基线提升划分质量，也能够谱初始化基础上进一步改进或持平。

7 偏差与失败原因分析

与原论文相比，本项目有两个重要偏差。第一，原始 Kernighan–Lin 方法是二路图划分算法，而本项目应用需要 k 个分区。因此我们把成对二分细化作为组件嵌入到 k 路划分框架中。

第二，本项目使用 NetworkX 中的成对 Kernighan–Lin 二分实现，而不是从零重写每一个底层细节。这一选择提升了鲁棒性，使项目能够把重点放在算法比较、改写和评估上。核心复现行为仍然是基于局部搜索的割值细化。

一致性实验也有一个限制：它测量跨 worker 通信量，但没有改变底层通信拓扑。因此，同一张图上不同划分的一致性误差几乎相同。这不是划分算法失败，而是仿真设计导致的结果。该仿真隔离考察的是通信放置成本，而不是改变数值平均过程本身。

8 运行方式

可以用以下命令复现实验：

```
python src/experiments.py --output-dir outputs --seed 7
```

该命令会将原始 CSV 文件写入 `outputs/partition_results.csv`，并将图像保存在 `outputs/figures/` 下。单元测试可以通过以下命令运行：

```
python -m pytest
```